Course 2 Notes: Advanced Learning Algorithms

Adrew Ng — Jesse Hernandez

DeepLearningAI/Stanford Online — 2025

My notes for the second course in a three part machine learning specialization course on coursera consisting of four learning modules.

1. Week 1: Neural Networks

1.1 Overview

- Key objectives
  - Introduction to Neural Networks
  - Nueral Network Training
  - Practical advice on building ML systems
  - Decision Trees
- Background context
  - Building Neural network from scratch and with Tensorflow for inference and training

1.1.1 Neurons as inspiration

Note

- The origins of neural networks were to build algorithms that mimicked the brain
- Re-popularized in 2005 in the category of Deep Leaning
- Initially successful in speech recognition then image recognition followed by Natural Language Processing (NLP) and now used in many more applications
- Biological neurons function by taking many electrical impulses as inputs and outputs the response to send to the next neuron until it reaches the brain
- Neural Networks saw huge performance improvements particularly with Big Data,e.g., performance scales with amount of data,

1.2 Introduction to Neural Networks

Definition

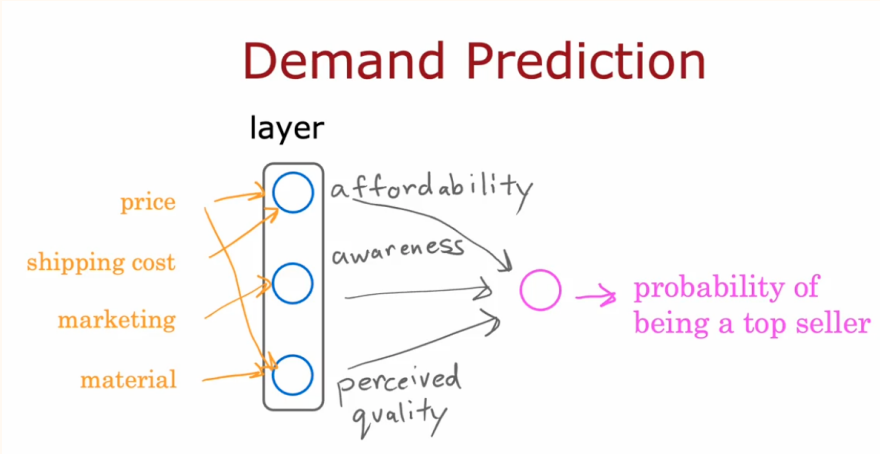
A **Neural Network** is a type of machine learning inspired by how the brain works using layered structure of interconnected nodes or 'neurons' to process data, recognize patterns and make predictions.

Example

Demand Prediction example using neural network.

- We are trying to predict the probability of being a top seller for clothing product
- The inputs or features to the model are price, shipping cost, marketing, and material (input layer)
- The criteria (**activations**) are affordability, awareness, and perceived quality (forms the nodes of our neural layer or **hidden layer** that outputs **activation values**)
- Relevant features map out to different nodes to access its criteria ranking, in practice the network will find the best features to use (analogous to automated feature engineering)
- All outputs from the layer is fed to another node (output layer) to process the probability of being a top seller
- the independent nodes each run independent logistic regression (input model) algorithms to output probabilities
- Simple to vectorize inputs and outputs of to run through layers and come to a prediction, here $\vec{x}$, $\vec{a}$ are the inputs and activation values respectively,

$$\vec{x} \rightarrow \textbf{(hidden layer)} \rightarrow \vec{a} \rightarrow \textbf{(output layer)} \rightarrow a$$



Note

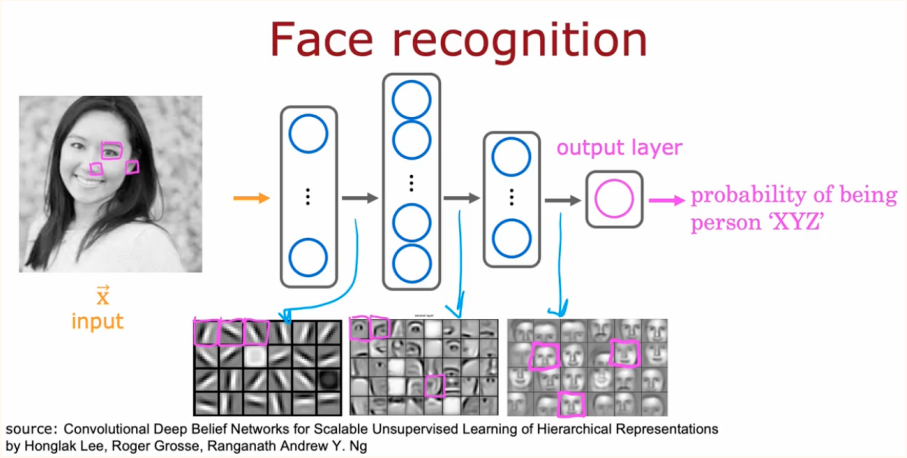
- Depending on the **neural network architecture** there can be many hidden layers with different number of nodes in each (**multilayer perceptron**)
- choosing how to construct the neural network depending on what problem you have can affect performance and accuracyand will be discussed in week 3

1.2.1 Recognizing Images

Example

Face recognition problem is to identify the person in the image using the pixel data of the image

- The image data is read as a $m \times n$ matrix of pixel intensities
- The input features are then these values rolled out into a vector of $m \times n$ length,e.g., a 1000x1000 pixel image will have 1 million pixel intensity features as input to the network
- Shown below is how many hidden layers may naturally break the task into pieces at each layer with different imgae scopes and aggregate the information to get a probability for being person XYZ
  1. Layer 1 may look for edges or shapes (fine scope)
  2. Layer two may take layer 1 input to find facial structures, e.g., eyes,nose, etc (medium scope)
  3. Layer 3 may agregate the facial features to form full facial images (large scope)



Note

The neural network can be tasked to find cars and the layers will automatically learn to search for car features.

1.3 Neural Networks in action

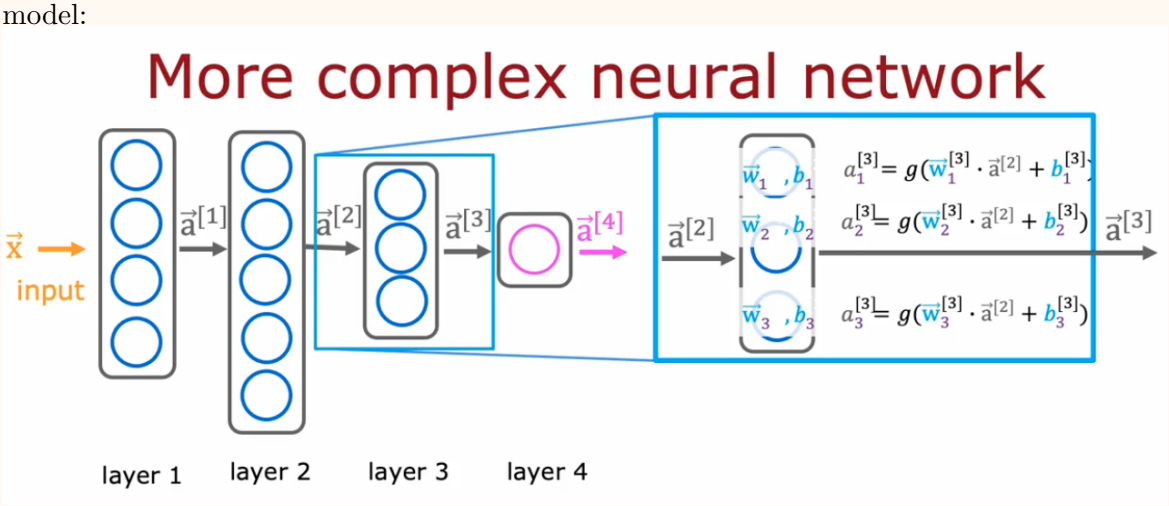
Definition

The **neural network layer** is composed of many nodes $(n_1 \dots n_i)$ taking in all the features $\vec{x}$ with parameters optimized for each node $(\vec{w}_i,\ b_i)$ from the set of features and outputs the activation value for each node a_i, which forms the activation vector(hidden layer output) $\vec{a}$.

- Each node is running a logistic regression model such that $a_i = g(z)$ where the definitions for the regression model z (linear or polynomial) is transformed using a sigmoid $(g(z))$ function discussed in course 1.
- For multi-layer neural networks we will use the superscript notation $\vec{a}^{[j]}$ to denote the j^{th} layer
- The activation values of each layer $\vec{a}^{[j]}$ then become the input values for the subsequent layer

Example

An example of a more complex neutral network may look like this (below), where the calculation for the third hidden layer using a logisitc regression transforamtion of a linear regression model:



- The third layer calculations are using the second layer outputs: $a_i^{[3]} = g(\vec{w}_i^{[3]} \cdot \vec{a}^{[2]} + b_i^{[3]})$

This can be generalized for logistic regression with a linear regression model as:

$$a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}), \tag{1}$$

where l is the layer number and j is the node number. Here $g(z)$ is the **activation function** and can take other forms but is a sigmoid function here.

1.3.1 Inference: making predictions

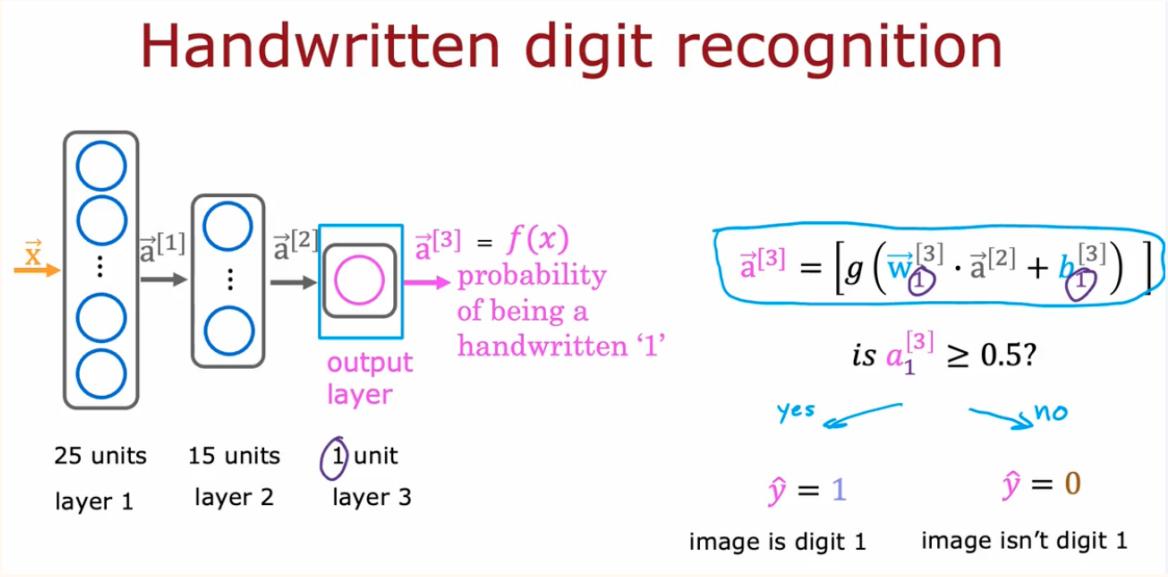
Definition

Forward propagation is a machine learning algorithm used with neural networks to make predictions by propagating the results forward through the hidden layers to the output value (see example below)

- Forward propagation can be used to predict values using an existing neural network with tuned parameters, in the next lesson we will learn to train our own neural network

Example

Handwritten digit recognition example is shown below where we want to find teh porbability of the image being a one and take an 8 x 8 pixel image as input with 2 hidden layers having 25 and 15 nodes



- The image above shows only the computation for the final output layer, more generally each activation vector is solved:

$$\vec{a}^{[l]} = \begin{bmatrix} g(\vec{w}_1^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}) \\ \vdots \\ g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]}) \end{bmatrix} \tag{2}$$

1.4 Implementations

– Neural network implementation in Tensorflow (Lab 1).
  – Neural network implementation from scratch using Numpy (Lab 2)
  – Matrix implementation for efficient vectorization for implementation

2. Week 2: Neural Network Training